

DASHBOARD

Architecture and technical benchmark document

Adams Kamate & Mohammad Amin Hammami

Justification of technology choices and implementation plan

Table of contents

1	Project requirement 14 constraints identified (C1 to C14)
2	Benchmarking and justification Front, back, base, cache, auth, API, deployment
3	Architecture applicative Components

1. CONSTRAINTS

Project requirements recap (1/2)

Before we justify anything, we need to list the project’s requirements. Every technical choice will be linked to one of the constraints.

#	Project requirements	Technical choice
C1	docker-compose build and docker-compose up mandatory	The entire stack must be containerizable
C2	Server on port 8080 responding to GET /about.json	HTTP backend exposing JSON with a mandated structure
C3	Sign-up + account confirmation before access	Internal auth (email/password), confirmation email
C4	Authentication (credentials or OIDC)	JWT / session
C5	Authorization via OAuth 2.0 for external APIs	Manual implementation of the OAuth2 flow (libraries not allowed)
C6	Link the third-party account to a platform user	User / service / tokens join table
C7	(1 + X) services and (3 × X) widgets for X students	Extensible architecture: no redesign needed for each added service
C8	Each widget must have configurable parameters	Storage of heterogeneous parameters per instance
C9	Timer: each instance reports its refresh rate	Scheduled task / polling system per instance
C10	Add / reconfigure / move / delete an instance	Full CRUD + position persistence
C11	Two instances of the same widget with different configs	Configuration belongs to the instance, not to the widget type
C12	Responsive interface, good UX, accessibility	Front-end framework with a mature UI ecosystem
C13	Personal data and sensitive credentials	Encrypted tokens, hashed passwords, no client-side secrets
C14	Complete README + professional justification	This document + ongoing documentation

Frontend : Next.js (React) + Tailwind CSS

Criteria	Next.js (React)	React seul (Vite)	Vue.js	Angular
Weight	Average	Light	Very light	Heavy
Learning curve	Easy if you know react	Easy	Easy	Hard, needs experience
Community & documentation	Massive	Massive	Good but smaller	Enterprise, Google libraries
Team maintainability	Very good	Good, but everything has to be structured manually	Good	Very if good of mastered
Built-in routing	Yes	No (external lib)	No (external lib)	Yes
Server routes (OAuth callback)	Yes	No	No	No
Recommended by the project brief	Yes (React)	Yes (React)	Yes	Yes

Choice: Next.js; the deciding factor is server routes, which allow the OAuth callback to be handled without exposing any secrets to the browser (C13).

Styling: Tailwind CSS, recommended by the project brief, rather than Bootstrap (generic visual identity) or plain CSS (too slow to satisfy C12).

Frontend : why Next.js (React) over the alternatives

1

Over Angular

The learning curve and overhead of the framework are disproportionate to the size of the project. The project brief explicitly recommends simple solutions and warns against over-engineering ("Focus on clear, minimal solutions").

2

Over Vue

Vue is lighter and faster, but its community and documentation are smaller. On a project with a tight deadline and OAuth and third-party API integration challenges, the amount of available resources matters more than a few kilobytes of bundle size.

3

Over React (alone)

Next.js provides native routing and, above all, server routes, which are very useful for handling OAuth callbacks without exposing any secrets to the browser (C13). This is the deciding difference between the two.

General principle : when functional constraints are equal, the tie-breaker is the security of the OAuth flow, followed by the volume of available resources to help the team get unblocked.

Backend : Node.js (Express) vs Go vs Python

Criteria	Node.js (Express)	Go	Python (Django/FastAPI)
Execution speed	Moderate	The fastest	The slowest
Community	Massive	Very large and active	Big
Build time	Fast	Fast	Fast
Recommended by the project brief	Yes	Not listed	Yes
A single language for front end and back end	Yes (TypeScript)	No	No
JSON handling (C2)	Native	Structs + tags, verbose	Native
Concurrent calls to N APIs (C9)	Very good (event loop, I/O-bound)	Excellent (goroutines)	Good with asyncs
Hand-written OAuth2 (C5)	Native fetch is enough	net/http is enough	httpx / requests
Development speed	Fast	More verbose	Fast
Memory / CPU usage	Low to moderate	Very low usage	High

Choice : Node.js + Express, over Go and Python.

Backend : the project's real bottleneck

The limiting factor is not performance: the expected traffic is that of a keynote demo, not large-scale production. The limiting factor is the volume of features to deliver (C7: up to $(1+X)$ services and $(3 \times X)$ widgets, each with its own API integration and hand-written OAuth flow).

Over Go

Go would win where raw performance and concurrency are critical. Here, the workload is I/O-bound (waiting for responses from external APIs), an area where Node's event loop is more than sufficient; the advantage of goroutines is real but yields no noticeable gain at this scale. On the downside: a second language to master, a more verbose syntax, and its absence from the list of recommended technologies.

Over Python

Its data/analytics ecosystem (pandas, numpy) is its main strength, yet this project performs no statistical computation: it aggregates and displays data from third-party APIs. That advantage is useless here, while the costs (an additional language, lower performance) remain.

In favor of Node

A single language across the entire stack allows TypeScript types to be shared between front end and back end (a Widget interface defined once), which inherently reduces integration errors. Native JSON handling makes generating about.json (C2) trivial.

Database PostgreSQL

Criteria	PostgreSQL	MySQL	MongoDB
Typed and indexable JSON support	JSONB, excellent	JSON basic	Natif (document)
Users / services / widgets relationships	Excellent	Excellent	Weak (no native joins)
Complex analytical queries	Excellent	Correct	Limited
ACID compliance	Strict	Good	Weaker
Extensions	A lot	Limited	—
Official Docker image (C1)	Yes	Yes	Yes

Over MySQL

C8/C11 require heterogeneous parameters per instance ({city: "Paris"} vs {link, number}). JSONB stores and queries these variable structures in a typed, indexable column; MySQL would require a cumbersome EAV table or non-queryable text JSON.

Over MongoDB

The model is fundamentally relational: a user subscribes to N services, each service exposes N widgets, and each widget has N instances. PostgreSQL provides both relational features AND JSON; MongoDB would provide only JSON.

Explicit decision: no time-series database (InfluxDB / TimescaleDB). The project brief requires no historical data: widgets display the current state of a source, not its stored evolution over time. Adding one would be over-engineering and one more Docker service to maintain for nothing.

Cache and task queue: Redis + BullMQ

Criteria	Redis	Memcached	RabbitMQ
In-memory cache	Yes	Yes	No
Task queue / scheduled jobs	Yes	No	Yes
Rich data structures	Yes	Key-value only	—
Optional persistence	Yes	No	Yes
Additional Docker services	1 (covers both roles)	1 (cache only)	1 (jobs only)

Why Redis

The project has two distinct needs: caching responses from external APIs and scheduling Timer refreshes (C9). Redis covers both with a single container, whereas Memcached or RabbitMQ would require adding a second one. Fewer services means a simpler docker-compose.yml (C1).

Caching is a necessity

The targeted APIs (GitHub, Steam, weather) enforce quotas. Without caching, ten users displaying the same widget would trigger ten identical calls and hit the rate limit.

Why BullMQ

Celery requires a Python runtime, which is inconsistent with a Node backend. node-cron handles neither job persistence (everything is lost when the container restarts) nor error recovery. BullMQ builds on the Redis instance already in place and supports repeatable jobs per instance.

Authentication and authorization

Mecanism	Role in the project brief	Technical choice
Authentication (C3, C4)	Identify the platform user	Email + hashed password (bcrypt/argon2) + JWT
Authorization (C5, C6)	Allow widgets to access external APIs	OAuth 2.0 Authorization Code, implémenté manuellement

Major constraint: no library may implement OAuth on your behalf; Passport.js, NextAuth, and simple-oauth2 are excluded. Only a generic HTTP client (native fetch) is used.

The flow to implement by hand

- 1 Generate a random state parameter (anti-CSRF), stored server-side or in a Redis session
- 2 Redirect to the provider's authorization URL (client_id, redirect_uri, scope, state)
- 3 Receive the callback and verify that the state matches the one issued
- 4 Server-to-server POST to the token endpoint: exchange the code for an access_token
- 5 Store the encrypted token in the database, linked to the platform user (C6)
- 6 Refresh the token on expiry using the refresh_token

Security considerations (C13)

- The client_secret never passes through the browser: the code exchange happens exclusively server-side.
- Tokens in the database are encrypted (AES-GCM with an application key), never stored in plain text.
- JWT over server-side sessions: stateless, no additional store required, and kept in an httpOnly cookie to resist XSS.

REST API and Docker Compose deployment

Choice: REST, over GraphQL and WebSockets

Over GraphQL

The project brief already mandates a REST endpoint with a fixed structure (GET /about.json, C2). Introducing GraphQL alongside it would mean maintaining two API paradigms in parallel, for no benefit at this scale.

Over WebSockets

The project brief explicitly describes a Timer-based polling mechanism. WebSockets would address a real-time push requirement that isn't asked for; they should be kept as a bonus, not as the foundation.

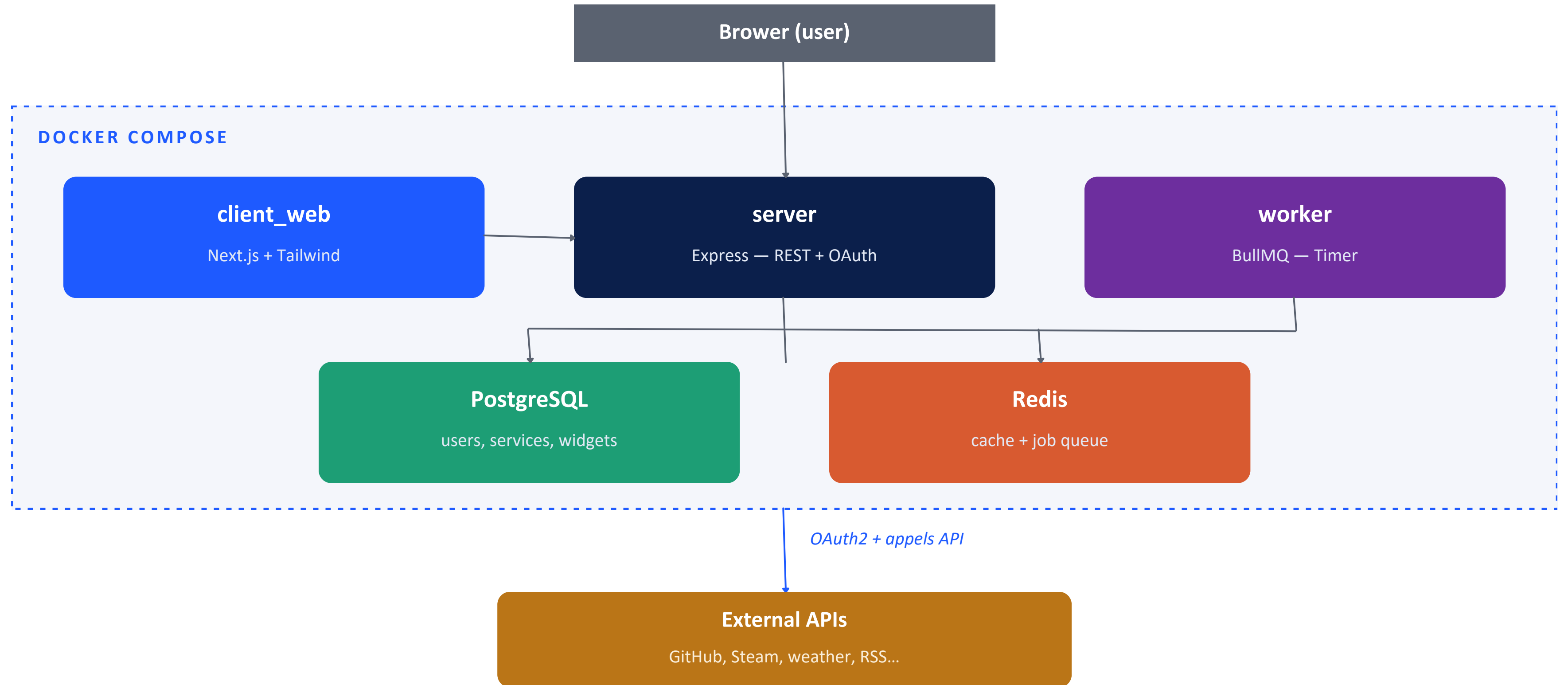
Deployment : Docker Compose (mandated, C1 and C2)

Service	Image / build	Port	Role
server	build backend	8080	API REST, OAuth, about.json
client_web	build frontend	8081	Interface Next.js
db	postgres:16-alpine	5432	Persistent data
redis	redis:7-alpine	6379	Cache + job queue
worker	build backend	—	Timer job execution

Key considerations

- Named volumes for PostgreSQL; otherwise, data is lost on every docker-compose down.
- depends_on with healthchecks: the backend must wait until PostgreSQL is actually ready, not just started.
- All secrets (OAuth client_id/secret, JWT key) in environment variables, never hard-coded or committed; provide a documented .env.example.

Component overview



Data model

Six tables are enough to cover all the functional constraints.

Table	Main columns	Constraint
users	id, email, password_hash, is_verified, verification_token, role, created_at	C3, C4
services	id, name, requires_auth (bool), oauth_config	C7
user_services	user_id, service_id, access_token (chiffré), refresh_token, expires_at	C5, C6, C13
widget_types	id, service_id, name, description, params_schema (JSONB)	C2, C8
widget_instances	id, user_id, widget_type_id, params (JSONB), refresh_rate, position_x/y, width, height	C8–C11
widget_cache	instance_id, data (JSONB), fetched_at	C9

The most important architectural point: the separation between widget_types and widget_instances

- widget_types describes what a widget can do (name, possible parameters); this is what directly feeds about.json (C2).
- widget_instances describes a configured occurrence on a user's dashboard, with its own params.
- This is exactly what satisfies C11: two instances of city_temperature, one with {city: "Paris"} and the other with {city: "Tokyo"}, display different data at the same time.

Extensible service architecture

C7 requires up to $(1+X)$ services and $(3 \times X)$ widgets. If each is coded ad hoc, duplication explodes, which the project brief explicitly forbids ("a simple, scalable architecture with no code duplication").

Solution: a shared contract (Provider pattern)

```
interface ServiceProvider {
  name: string;
  requiresAuth: boolean;
  getOAuthConfig(): OAuthConfig;
  widgets: WidgetDefinition[];
}

interface WidgetDefinition {
  name: string;
  description: string;
  params: { name: string;
    type: 'string' | 'integer' }[];
  fetch(params, token?): Promise<WidgetData>;
}
```

Direct benefits

- 1 about.json is generated automatically**
by iterating over the registry: no list to maintain by hand, no risk of getting out of sync (C2)
- 2 Adding a service = creating a file**
in services/ and registering it: no changes to the application core
- 3 The Timer is generic**
it calls fetch() without knowing which service it's dealing with
- 4 The frontend is generic**
it renders the configuration form from the params field returned by the API

This is what turns adding widgets from a redesign effort into mechanical work that can be parallelized across team members.

How the timer works

1

The user adds an instance

with refresh_rate = 300s

2

The backend registers a repeatable job

BullMQ (every: 300000) tied to instance_id

3

On each trigger, the worker

loads the instance and its params, retrieves the OAuth token if needed, calls provider.fetch(params, token), and writes the result to the cache

4

The frontend queries GET /widgets/:id/data

reads from the cache, immediate response

5

Reconfiguration -> the job is updated

instance deletion -> the job is removed

The key point: the frontend never triggers a call to an external API. It always reads from the cache, which is what guarantees fast display and protects against rate limits.

Project structure

```
dashboard/
├── docker-compose.yml
├── .env.example
├── README.md
├── server/
│   ├── Dockerfile
│   └── src/
│       ├── index.ts           # Express, port 8080
│       ├── routes/
│       │   ├── about.ts      # GET /about.json
│       │   ├── auth.ts       # register, login, verify
│       │   ├── oauth.ts      # authorize + callback
│       │   ├── services.ts   # souscriptions
│       │   └── widgets.ts    # CRUD des instances
│       ├── services/         # un fichier par service
│       │   ├── registry.ts
│       │   ├── weather.ts
│       │   ├── github.ts
│       │   └── rss.ts
│       ├── jobs/             # worker BullMQ (Timer)
│       ├── db/               # migrations + accès
│       └── lib/              # jwt, crypto, oauth
├── client_web/
│   ├── Dockerfile
│   └── src/
│       ├── app/              # routes Next.js
│       └── components/widgets/
└── bonus/                   # exigé par le sujet
```

What this structure guarantee

- An isolated services/ folder**
Each service is a self-contained file: team members work in parallel without merge conflicts.
- Routes separated by domain**
auth, oauth, services, and widgets never mix; the OAuth logic stays in one place.
- lib/ for cross-cutting concerns**
JWT, token encryption, and the OAuth helper are reused by all providers.
- bonus/ from the start**
The project brief requires all bonus files to be in this folder; creating it right away avoids a last-minute cleanup.

Implementation plan: Phases 0 and 1

Guiding rule: never build a layer that depends on an untested layer, and tackle the highest technical risk as early as possible.

Phase 0: Foundations (before any business logic)

#	Task	Acceptance criterion
0.1	Initialize the repo, project structure, linter, and commit conventions	The repo is clean and shared
0.2	Write the docker-compose.yml with the 5 services and healthchecks	docker-compose up starts everything without errors
0.3	Express skeleton on port 8080	The container responds
0.4	GET /about.json with hard-coded data, in the exact format specified by the project brief	curl localhost:8080/about.json returns the expected structure

Why first: about.json on port 8080 is the project's automated validation criterion. Getting it out immediately, even with dummy data, eliminates any risk at submission time.

Phase 1: Data and authentication

#	Task	Acceptance criterion
1.1	Complete PostgreSQL schema + migration system	Migrations apply cleanly to an empty database
1.2	Sign-up with password hashing	The password is never stored in plain text
1.3	Account confirmation by email (C3)	An unconfirmed account cannot log in
1.4	Login + JWT + route protection middleware	A protected route returns 401 without a valid token

Implementation plan: Phases 2 and 3

Phase 2: Hand-written OAuth (the main technical risk)

#	Task	Acceptance criterion
2.1	Generic OAuth helper: state, authorization URL	The generated URL complies with the provider's documentation
2.2	Callback: state verification, code → access_token exchange	The token is retrieved without using an OAuth library.
2.3	Encryption and storage of the user-linked token (C6, C13)	The token stored in the database is unreadable without the key.
2.4	Automatic refresh of the expired token	A widget continues to function after token expiration.

Why so early: this is the most uncertain part (manual implementation required, with behavior varying across providers). It’s better to tackle the difficult parts in week 1 than in the final week.

Phase 3: First Complete Functional Cycle (the Vertical Slice)

#	Task	Acceptance criterion
3.1	Define the ServiceProvider interface and registry	The extensible architecture is in place.
3.2	A weather service without authentication with a configurable widget	The widget returns real-world data.
3.3	Dynamically generate about.json from the registry	The JSON automatically reflects the providers.
3.4	Full CRUD for widget instances (C10)	Create, reconfigure, move, and delete operations work.
3.5	Two instances of the same widget with different configurations (C11)	The two instances return distinct data.

Validating the entire flow (provider → database → API → configuration) with a single widget helps uncover architectural issues while there is still only one case to fix.

Implementation Plan: Phases 4 and 5

Phase 4: Timer and Cache

#	Task	Acceptance criterion
4.1	Set up Redis + BullMQ and start the worker	The worker consumes the queue.
4.2	Create, update, and delete the job based on the instance's refresh rate.	Changing the refresh rate actually changes the frequency.
4.3	Write results to the cache, read them through the API.	The response is served from the cache.
4.4	Error handling: API unavailable, rate limit, invalid token	A widget in error does not prevent the others from functioning.

Phase 5 : Frontend

#	Task	Acceptance criterion
5.1	Registration, confirmation, and login pages	The complete flow works in the browser.
5.2	Service subscription (OAuth linking button)	The third-party account is linked to the platform account.
5.3	Configuration form generated from params	A new backend widget automatically displays its form without any frontend code.
5.4	Grid: add, move, resize, and delete (C10)	Positions are persisted after reloading.
5.5	Responsive design, contrast, keyboard navigation, and ARIA labels (C12)	Usable on mobile devices and with keyboard navigation.

Testing the API with curl/Postman before connecting the frontend avoids debugging two layers simultaneously.

Implementation Plan: Phases 6 and 7

Phase 6: Scaling Up (Parallelizable Across the Team)

#	Task	Acceptance criterion
6.1	Implement the remaining services until reaching (1+X).	All of them appear in about.json.
6.2	Implement the widgets up to (3×X), all with parameters (C8).	No widget without configurable parameters.
6.3	User administration section	An admin can list and manage user accounts.

Work is divided by service among team members, rather than by technical layer. Each person works on an isolated file, which reduces merge conflicts.

Phase 7: Finalization and Deliverables

#	Task	Acceptance criterion
7.1	Security audit: secrets outside the repository, encrypted tokens, input validation	No secrets committed, no injection vulnerabilities.
7.2	Repository cleanup; bonus files in bonus/	The repository meets the delivery requirements.
7.3	Complete README.md (C14)	An external developer can run the project without assistance.
7.4	Dry run: clone on a clean machine → docker-compose up	The application starts successfully on the first attempt.
7.5	Keynote + demo video	The demo does not depend on a live environment.

Identified Risks

Risk	Impact	Mitigation
Manual OAuth implementation takes longer than expected.	It blocks all authenticated services.	Addressed in Phase 2, very early on, starting with a single provider.
Rate limit reached on a third-party API	Widgets not functioning during the demo	Redis cache introduced in Phase 4, with a mandatory minimum refresh rate
Widget quota not met	Direct penalty on the grade (C7)	The Provider pattern makes adding new services straightforward; plan for them from the start.
Secrets committed to the repository	Security vulnerability, penalty (C13)	.env in .gitignore from the initial commit, with a documented .env.example.
docker-compose up fails on a clean machine.	Project rejected during evaluation.	Dry run in Phase 7, health checks and volumes from Phase 0.
README written at the end and rushed.	Explicitly required deliverable (C14)	Document continuously, immediately after each feature.

The first three risks are addressed by the structure of the plan itself: the most uncertain one (OAuth) is tackled in Phase 2, while the Provider pattern introduced in Phase 3 mitigates the risk of falling short of the required service quota.

SUMMARY

Summary of choices

Layer	Selected Approach	Rejected in favor of this approach	Decisive reason
Frontend	Next.js + Tailwind	React alone, Vue, Angular	Server-side routes for OAuth callbacks, maximizing community support
Backend	Node.js + Express	Go, Python	Single language for frontend/backend, enabling faster development on an I/O-bound project.
Data-base	PostgreSQL	MySQL, MongoDB	JSONB for heterogeneous parameters + relational model
Time-series	None	InfluxDB, TimescaleDB	No temporal history required by the project requirements.
Cache + jobs	Redis + BullMQ	Memcached, RabbitMQ, Celery, node-cron	A single container for two needs, persistent jobs.
API	REST	GraphQL, WebSockets	Format already mandated by the project requirements, polling is sufficient.
Auth	JWT + OAuth2 hand made	Passport.js, NextAuth (forbidden)	Explicit project requirement.
Deployment	Docker Compose	—	Mandated by the project requirements.

Each line is justified in the full document: what was discarded, and which specific project requirement the decision is based on.

Table of contents

1	Summary of the Project Requirements 14 extracted requirements (C1 to C14)
2	Benchmark and Rationale Front, back, base, cache, auth, API, deployment
3	Application Architecture Components, data model, Provider pattern, Timer
4	Implementation Plan 8 chronological phases with validation criteria
5	Risks and Mitigation What could cause the project to fail
6	Summary of choices One-page summary